GraphQL-Shield

Dual-Layer Protection of GraphQL APIs against Broken Object Level Authorization (BOLA)

[Student Name 1] [Student Name 2] [Student Name 3] [Student Name 4]

Guide: [Guide Name], [Designation]
Department of [Department], [College / University Name]

Review 3

Outline

1. Introduction
2. Problem Statement & Motivation
3. Objectives
4. Literature Survey
5. Research Gap
6. Proposed System
7. System Architecture / Methodology
8. Implementation
9. Results and Discussion
10. Conclusion
11. Future Scope
12. References
13. Research Paper Template

What is an API, and what is GraphQL?

- Apps (Instagram, banking, hospital portals) talk to their servers through an **API**.
- **GraphQL** is a modern API style made by Facebook: one address, and the app asks for *exactly* the data it wants.
  - Example: "give me my name, my documents, and the files inside my folder" – in one request.
- Flexible and fast – so it is now widely used.
- But the flexibility opens a new security hole.

In simple words

REST is like a fixed restaurant menu. GraphQL is like telling the waiter exactly what to bring. Very convenient – but the kitchen must still check that you only get *your* order.

The Threat: BOLA

- **BOLA = Broken Object Level Authorization.**
- The server knows *who* you are (you logged in), but forgets to check whether the item you asked for is *yours*.
- Attacker simply changes a number: `resource(id: 101)` → `resource(id: 102)` and reads someone else's data.
- Ranked **#1 risk** in the OWASP API Security Top 10 (2023).
- Firewalls cannot see it – the request looks completely normal.

In simple words

Like a hotel key card that opens every room if you just type a different room number. You are a real guest – but you should only open your own room.

Our Solution in One Line

- **GraphQL-Shield**: a security gateway placed in front of any GraphQL app.
- It checks ownership at **two independent points**:
  - **Layer 1** – before the request runs: "does this request name anything that isn't yours?"
  - **Layer 2** – while it runs: "is every item actually being returned or changed yours?"
- Novel idea: Layer 2 also **reports what Layer 1 missed**, so the system measures and repairs its own gaps.

In simple words

Two security guards: one at the hotel entrance checks your request, one in the corridor checks every door you open – and writes down anything the entrance guard missed.

Problem Statement

- **Problem:** Design and implement a runtime defence that stops logged-in users of a GraphQL API from reading or changing objects they do not own – in every form GraphQL allows – while:
  - keeping legitimate access working,
  - adding very little delay,
  - and showing where the defence itself is incomplete.

Form of attack	Example	ID visible in request?
Direct	resource(id: 102)	Yes
Nested	user(id:101){resource(id:102)}	Yes, deeper
Disguised	variable, alias, fragment, batch	Only after decoding
Relationship	folder(201){children{...}}	No – hidden
Write	updateResource(id:102, ...)	Yes

Motivation – Why This Matters

- BOLA directly causes data breaches of personal, medical and financial records.
- Existing tools only **test** for BOLA before release, or need a hand-written rule for **every field** – one forgotten rule silently leaks data.
- Checking only the request cannot see items reached through links (folder → children).
- Wrongly placed blocks in GraphQL can wipe out the user's **own** data too (null propagation).
- No existing defence tells the developer **where it is weak**.

In simple words

Current locks either get checked once at the factory, or need a separate lock on every drawer. We want a lock that checks itself and tells you when a drawer was left open.

Objectives

- **Block every form of BOLA** – direct, nested, disguised, relationship and write attacks.
- **Defence in depth:** two independent layers (before and during execution).
- **Cross-layer evidence:** Layer 2 records every object Layer 1 never checked.
- **Blind-spot prediction:** XGBoost model with SHAP explanations, trained from that evidence.
- **Self-repairing configuration:** verified fixes, applied only with admin approval.
- **Formal security properties** (Safety, Non-interference, Write safety, Availability) tested automatically.
- **Close side channels** – same error and same timing for "not yours" and "does not exist".
- **Secure accounts** (hashed passwords, lockout, token revocation) and **low overhead** (~1 ms).

Literature Survey

Work	Approach	Limitation for us
OWASP API Top 10 (2023) [1]	Ranks BOLA as #1 API risk	Guidance only
GraphQLer (Tsai et al., 2025) [2]	Replays GraphQL requests as other users	Test-time, no runtime blocking
AuthProbe (Barach, 2026) [3]	Multi-identity BOLA testing from OpenAPI	REST only, test-time
Santos Filho et al. (2025) [4]	OpenAPI $\rightarrow$ Petri nets attack models	REST only, detection
BolaZ (Wu et al., 2026) [5]	Static taint + runtime checks (Spring Boot)	Java REST, no self-measurement
Lambers et al. (2026) [6]	Taint analysis for graph APIs	Static only
graphql-shield library [7]	Permission rule per type/field	Many rules; can null own data
Hartig & Pérez; Cha et al. [8],[9]	GraphQL semantics, query cost	Not about ownership
Kaur (2026) [11]; CVE-2022-39275 [12]	100+ real BOLA bug reports; Saleor ID bug	Shows the threat is real

Key Observations from the Survey

- **Testing tools** find BOLA before deployment but give no protection at runtime – and most target REST, not GraphQL.
- **Enforcement tools** either inspect only the request (miss relationship links) or need one rule per field (leak when one is forgotten).
- **GraphQL research** studies query cost and semantics, not *who owns* the data.
- **Classic security principles** – reference monitor, complete mediation, non-interference – are well known but never combined into a self-checking GraphQL defence.

In simple words

Others built either a scanner (finds problems before opening day) or a rulebook (one rule per door). Nobody built a guard that also audits itself.

Research Gap

- **1.** No runtime defence covers **all** GraphQL BOLA forms (relationship links are missed).
- **2.** No defence **measures itself** – none uses one layer to check another's coverage.
- **3.** No **automatic, verified repair** of a wrong security configuration.
- **4. Availability ignored** – rule-based tools can delete the user's own data from the answer.
- **5. Side channels** – different errors or timings reveal which objects exist.
- **6.** No stated and **tested security properties**.
- **7.** Tied to **one framework**.

In simple words

GraphQL-Shield is designed to close each of these seven gaps.

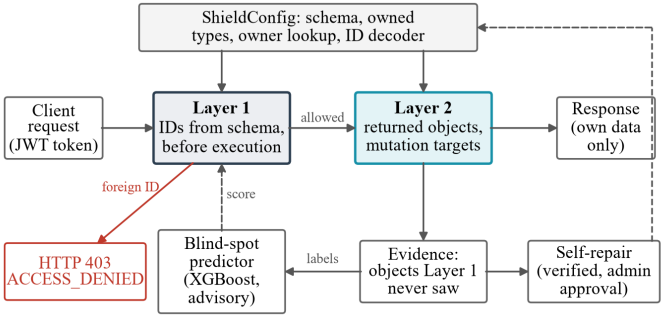
Proposed System: GraphQL-Shield

- Configured once with a small **ShieldConfig**: which data types are private, and how to find an item's owner.
- **No security code needed inside the app** itself.
- **Attacker model:** a real, logged-in user who tries to read or change other people's items.
- **Four guarantees** (for a normal user u):
  - **P1 Safety** – the answer never contains another user's object.
  - **P2 Non-interference** – changing other users' data never changes u's answer.
  - **P3 Write safety** – u can never modify or delete others' objects.
  - **P4 Availability** – u's own objects are always returned.

Main Components

- **Layer 1 – Entrance gate:** reads the request before it runs; any foreign ID → HTTP 403, nothing executes.
- **Layer 2 – Inside guard:** checks every object actually returned and every write target; removes foreign items.
- **Cross-layer evidence:** logs every object Layer 1 never saw → shows Layer 1's blind spots.
- **Blind-spot predictor:** XGBoost on 17 request features (advisory only).
- **Self-repair:** diagnoses the gap, proposes a fix, replays old traffic to verify it, admin approves.
- **Uniform denials**, **account protection**, **tamper-evident (hash-chained) audit log**, **live dashboard**.

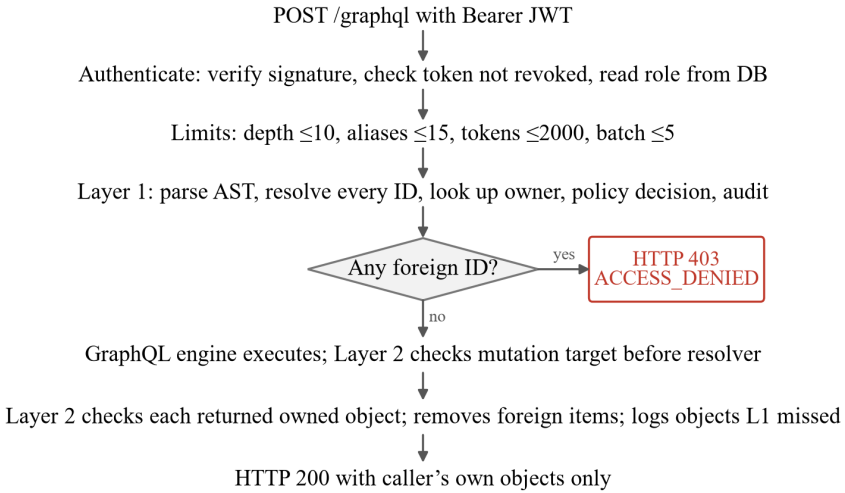
System Architecture



In simple words

Request → Layer 1 (block if foreign ID) → app runs → Layer 2 (remove foreign items) → only your data comes back. Disagreements between the layers feed the predictor and the self-repair module.

Methodology: Life of One Request



Methodology: How Each Part Works

- **Layer 1:** walks the parsed query, decodes every ID (variables, defaults, aliases, fragments, batches, Relay IDs), looks up the owner – one foreign ID rejects the whole request.
- **Layer 2:** runs inside the GraphQL engine; checks the real mutation target before the write; filters lists *before* GraphQL's null rules apply, so your own data survives.
- **Evidence:** Layer 1 marks what it checked; Layer 2 records anything unmarked = proof of a Layer 1 gap.
- **Self-repair:** a fix is offered only if replay shows it closes gaps and blocks no legitimate request.
- **Evaluation:** 6 research questions – ablation, fault detection, properties, comparison, latency, self-repair.

Implementation: Technology Stack

Component	Technology
Language / web	Python 3.13, FastAPI, Uvicorn
GraphQL	Strawberry, graphql-core, Graphene, Ariadne
Authentication	JWT (HS256), PBKDF2-SHA256 passwords, lockout
Database	SQLite with connection pooling
Machine learning	XGBoost, scikit-learn, SHAP
Testing	pytest, Hypothesis – 427 automated tests
Comparison baseline	Node.js + graphql-shield 7.6.5
Front-end / DevOps	HTML/JS dashboard, Docker, GitHub Actions

In simple words

Two separate apps are protected (a document app and a clinic app with patient records) – the clinic app has zero security code of its own.

Implementation: Live Demo Scenarios

Demo (user 101 attacks)	Request	Outcome
Legitimate access	resource(id: 101)	200 OK
Direct BOLA	resource(id: 102)	403 – Layer 1
Nested BOLA	user(101){resource(102)}	403 – Layer 1
Unauthorised write	updateResource(102, "Hacked")	Blocked, DB unchanged
Relationship traversal	resource(201){children}	200 – foreign child removed
Admin override	admin reads resource(102)	200 – logged

In simple words

Dashboard (/dashboard): run every attack with one click, switch layers off to see what each one stops, view the ML models, self-repair and the audit log.

Result 1: Which Layer Stops Each Attack?

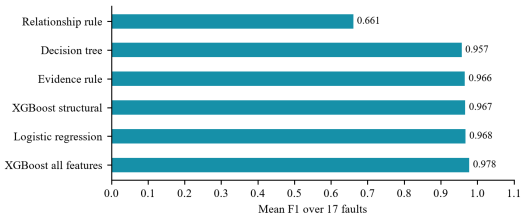
Attack	Both layers	L1 only	L2 only	None
Direct	L1	L1	L2	leaked
Nested lookup	L1	L1	L2	leaked
Inline fragment	L1	L1	L2	leaked
Relationship	L2	**leaked**	L2	leaked
Batch	L1	L1	L2	leaked
Update mutation	L1	L1	L2	leaked

In simple words

Layer 1 alone misses the hidden "children" attack; Layer 2 catches it. Together: zero leaks, and most attacks stopped before anything runs.

Result 2: The Evidence Finds Bugs

- **Real bug found:** Layer 1 ignored default values of variables – 585 requests slipped past it (Layer 2 still stopped them). Fixed.
- **17 bugs deliberately planted** in Layer 1, 5 runs each:
  - **17 / 17 detected**, **0 false alarms**.
  - A one-line rule predicts affected requests with F1 = 0.966.



In simple words

The value is in the evidence, not the fancy model – XGBoost is no better than simple logistic regression.

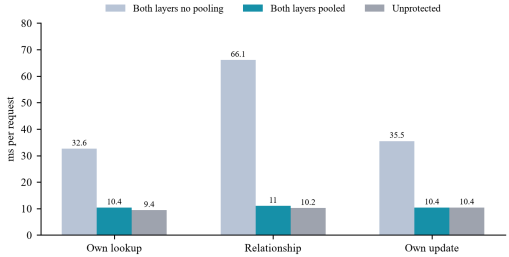
Result 3: Properties, Leaks and Comparison

- **P1–P4 hold** with protection on (Hypothesis, 120 examples each); P1–P3 fail with it off – proving the tests work.
- **4 hidden leak channels found and closed** (error messages, interfaces, counts, refused deletes).
- **Timing side channel closed:** 0.352 ms gap before fix → no measurable gap after.
- **vs. graphql-shield** (13 attacks, 5 normal requests):

Measure	GraphQL-Shield	graphql-shield
Attacks leaked	0 / 13	0 / 13
Legitimate requests broken	**0 / 5**	**1 / 5**
Configuration written	~4 entries + owner lookup	8 entries + 4 rules

Result 4: Speed and Self-Repair

- Protection costs **about 1 ms** per request (with connection pooling).
- **Self-repair:** 10 planted config mistakes $\times$ 3 runs:
  - **30 / 30** detected and fixed by the first verified repair
  - **33 / 33** wrong repairs refused
  - **0** legitimate requests blocked
- **Limitations:** apps and attacks written by us; synthetic ML data; owner-or-admin policies only.



Conclusion

- Checking ownership at **two independent points** stops BOLA in every form tested – including hidden relationship attacks.
- Layer 2's record of what Layer 1 missed **detected all 17 planted bugs** with no false alarms.
- The same evidence **repaired all 10 configuration mistakes** and refused all 33 wrong fixes.
- Side channels closed; own data never lost; ~**1 ms overhead**; works on 4 Python GraphQL frameworks.
- Demonstrates defence in depth, complete mediation and fail-safe defaults on a modern API.

In simple words

We built a lock that not only keeps intruders out, but also notices when it is weak and fixes itself – with an admin's approval.

Future Scope

- Test on real open-source GraphQL apps (e.g. Saleor) with independent attackers.
- Richer policies: sharing, teams, attribute-based access control.
- Suggest code patches, not only configuration fixes.
- Measure timing over real networks; production deployment with TLS, Redis, Kong / Envoy.
- Train ML models on real anonymised traffic.
- Port to Node.js (Apollo), Java and Go; add MFA / OAuth 2.0; stream logs to SIEM tools.

References (1/2)

[1] OWASP, "API Security Top 10 – 2023: API1 Broken Object Level Authorization," 2023.

[2] O. Tsai et al., "GraphQLer: Enhancing GraphQL security with context-aware API testing," arXiv:2504.13358, 2025.

[3] J. Barach, "AuthProbe: Specification-driven, multi-identity detection of BOLA," arXiv:2607.20574, 2026.

[4] A. Santos Filho, R. J. Rodríguez, E. L. Feitosa, "Automated BOLA attack detection in REST APIs through OpenAPI to colored Petri nets," Int. J. Inf. Security, 24(2), 2025.

[5] A. Wu et al., "Rethinking broken object level authorization attacks under zero trust principle," ACM TOSEM, 2026.

[6] L. Lambers et al., "Taint analysis for graph APIs focusing on broken access control," LMCS, 22(1), 2026.

[7] graphql-shield, npm package v7.6.5.

[8] O. Hartig, J. Pérez, "Semantics and complexity of GraphQL," WWW, 2018.

[9] A. Cha et al., "A principled approach to GraphQL query cost analysis," ESEC/FSE, 2020.

[10] E. Wittern et al., "An empirical study of GraphQL schemas," ICSOC, 2019.

[11] B. Kaur, "Broken object level authorization in the wild," arXiv:2605.25865, 2026.

[12] CVE-2022-39275: Saleor mutations did not validate object ID types, 2022.

References (2/2)

[13] GraphQL Foundation, "GraphQL specification: Handling field errors."
[14] J. P. Anderson, "Computer security technology planning study," ESD-TR-73-51, 1972.
[15] J. H. Saltzer, M. D. Schroeder, "The protection of information in computer systems," Proc. IEEE, 1975.
[16] J. A. Goguen, J. Meseguer, "Security policies and security models," IEEE S&P, 1982.
[17] W. M. McKeeman, "Differential testing for software," Digital Tech. J., 1998.
[18] Y. Jia, M. Harman, "An analysis and survey of mutation testing," IEEE TSE, 2011.
[19] K. Claessen, J. Hughes, "QuickCheck," ICFP, 2000.
[20] D. R. MacIver et al., "Hypothesis: A new approach to property-based testing," JOSS, 2019.
[21] A. Ratner et al., "Snorkel: Rapid training data creation with weak supervision," PVLDB, 2017.
[22] T. Chen, C. Guestrin, "XGBoost: A scalable tree boosting system," KDD, 2016.
[23] S. M. Lundberg, S.-I. Lee, "A unified approach to interpreting model predictions," NeurIPS, 2017.

Research Paper (IEEE Conference Template)

Item	Details
Title	GraphQL-Shield: Dual-Layer Enforcement Against GraphQL BOLA with Cross-Layer Coverage Evidence
Template	IEEE Conference (two-column)
Authors	[Author 1], [Author 2], [Author 3], [Author 4]
Keywords	GraphQL, API security, BOLA, access control, defense in depth, fault injection, property-based testing
Length	8 pages, 23 references, 2 figures, 7 tables
Files	paper/main.tex, paper/references.bib, paper/main.pdf

In simple words

The full work is written up as an 8-page IEEE two-column research paper, compiled from LaTeX.

Thank You

Questions?

Code: `https://github.com/Chinmayanand03/GraphQL-Shield-`